

**SIMATS ENGINEERING**  
**ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)**

**COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B**

**COURSE OUTCOME COVERED**

**CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)***

**Assessment Tool Weightage: 20%**

**QUESTIONS:5 Total Marks 20 Duration :60 Minutes Annexure A**  
**Pseudocode Writing Task (Algorithm Design)**

***Code of Conduct:***

***I R Dinesh-192421117 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.***

***Signature of the student:***

**Rubrics for Evaluation**

| Criteria                  | Weightage (%) | Level 4 (Exemplary)                                                   | Level 3 (Proficient)           | Level 2 (Developing)         | Level 1 (Beginning)             | Marks(100) |
|---------------------------|---------------|-----------------------------------------------------------------------|--------------------------------|------------------------------|---------------------------------|------------|
| Problem Understanding     | 20%           | Identifies all inputs, outputs, constraints accurately                | Minor missing elements         | Partial understanding        | Incorrect interpretation        |            |
| Algorithm Design          | 30%           | Complete, correct, and logically sound solution                       | Minor logical gaps             | Partially correct logic      | Incorrect approach              |            |
| Use of Control Structures | 20%           | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies           | Limited or basic usage       | Incorrect or missing constructs |            |
| Pseudocode Structure      | 10%           | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues | Difficult to follow in parts | Poorly structured and unclear   |            |

|              |     |                                          |                                                    |                                |                              |  |
|--------------|-----|------------------------------------------|----------------------------------------------------|--------------------------------|------------------------------|--|
| Completeness | 20% | Handles all cases;<br>optimized approach | Handles most<br>cases;<br>reasonable<br>efficiency | Handles<br>basic cases<br>only | Incomplete or<br>inefficient |  |
|--------------|-----|------------------------------------------|----------------------------------------------------|--------------------------------|------------------------------|--|

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

### 1. Real-Time Object Detection Pipeline

A system must:

- A. Capture video frames
- B. Detect objects using a trained model
- C. Draw bounding boxes and labels
- D. Display results in real time

Constraints:

- Must process frames efficiently
- Handle multiple objects

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

### 2. Image Difference Detection System

A system must:

- A. Read two images
- B. Compare them
- C. Highlight differences

Constraints:

- Handle noise
- Ensure accurate difference detection

Write pseudocode including preprocessing, comparison logic, and visualization.

### 3. Video Frame Extraction System

A system must:

- A. Read video input
- B. Extract frames at fixed intervals
- C. Save selected frames

Constraints:

- Efficient storage
- Avoid redundant frames

Write structured pseudocode for frame selection and saving logic.

### 4. Background Subtraction Pipeline

A surveillance system must:

- A. Read video stream
- B. Separate foreground from background
- C. Highlight moving objects

Constraints:

- Handle dynamic background

- Reduce noise

Write pseudocode including background modeling, subtraction, filtering, and display.

## 5. Edge Detection Pipeline for Real-Time Video

A system must:

- A. Capture video frames
- B. Apply edge detection
- C. Display results

Constraints:

- Real-time processing
- Maintain clarity of edges

Write pseudocode with preprocessing, edge detection, and visualization modules.

## 1. Real-Time Object Detection Pipeline

### Pseudocode

START

1. Load the trained object detection model.
2. Open the video camera.
3. WHILE camera is running:
  - a. Capture one video frame.
  - b. Resize the frame to a suitable resolution for faster processing.
  - c. Send the frame to the trained detection model.
  - d. Get detected objects, confidence scores, and bounding box coordinates.
  - e. FOR each detected object:
    - Draw a bounding box.
    - Display object label.
    - Display confidence score.
 END FOR
  - f. Display the processed frame.
  - g. Check for exit key.
    - IF exit key is pressed:
      - BREAK
 END IF
4. Release the camera.

5. Close all display windows.

END

### Modules

**Input → Preprocessing → Object Detection → Visualization → Display**

### Justification

- Resizing improves processing speed.
- The model can detect multiple objects in each frame.
- Bounding boxes and labels make the result easy to understand.
- Processing frames continuously provides real-time detection.

---

## 2. Image Difference Detection System

### Pseudocode

START

1. Read Image1 and Image2.
2. Check whether both images are successfully loaded.
3. Resize both images to the same size.
4. Convert both images to grayscale.
5. Apply Gaussian blur to both images to reduce noise.
6. Compare Image1 and Image2 using absolute difference.
7. Apply a threshold to the difference image.
8. Apply morphological operations to remove small noise and fill gaps.
9. Find contours of the detected differences.
10. FOR each significant contour:  
    Draw a bounding box around the difference.  
    END FOR
11. Display:  
    Original Image1  
    Original Image2  
    Difference Image
12. Save the final difference image if required.

END

## Modules

**Input → Grayscale → Noise Removal → Difference → Threshold → Morphology → Contours → Visualization**

## Justification

- Gaussian blur reduces noise.
- Absolute difference identifies changed pixels.
- Thresholding separates actual changes from small variations.
- Morphological filtering reduces false detections.

---

## 3. Video Frame Extraction System

### Pseudocode

START

1. Open the input video.
2. Get the video frame rate.
3. Define the required frame interval.
4. Set frame\_count = 0.
5. Set saved\_count = 0.
6. WHILE video has frames:
  - a. Read the next frame.
  - b. IF frame cannot be read:  
    BREAK  
    END IF
  - c. Increase frame\_count.
  - d. IF frame\_count MOD interval = 0:  
  
    Check whether the frame  
    is sufficiently different  
    from the previously saved frame.  
  
    IF frame is not redundant:  
        Save the frame.  
        Increase saved\_count.  
    END IF
7. Release the video.

8. Display the number of saved frames.

END

### Modules

**Video Input → Frame Counting → Interval Selection → Redundancy Check → Save Frame**

### Justification

- Fixed intervals reduce the number of stored frames.
- Redundancy checking avoids saving nearly identical frames.
- Only selected frames are stored, reducing storage requirements.

---

## 4. Background Subtraction Pipeline

### Pseudocode

START

1. Open the surveillance video.
2. Initialize a background model.
3. WHILE video is running:
  - a. Capture a frame.
  - b. Resize the frame if necessary.
  - c. Apply background subtraction  
to separate foreground from background.
  - d. Generate a foreground mask.
  - e. Apply thresholding to remove  
weak foreground regions.
  - f. Apply morphological opening  
to remove noise.
  - g. Apply morphological closing  
to fill small holes and gaps.
  - h. Find contours of foreground objects.
  - i. FOR each significant contour:  
    Draw a bounding box  
    around the moving object.  
END FOR
  - j. Display:  
    Original Frame

Foreground Mask  
Detected Moving Objects

4. Update the background model  
continuously to handle dynamic backgrounds.

5. IF exit key is pressed:

BREAK

END IF

6. Release video and close windows.

END

### Modules

**Video → Background Model → Subtraction → Threshold → Morphology → Contours → Visualization**

### Justification

- Adaptive background modeling handles gradual background changes.
- Thresholding removes weak noise.
- Opening removes small unwanted regions.
- Closing connects broken foreground regions.
- Contours identify moving objects.

---

## 5. Edge Detection Pipeline for Real-Time Video

### Pseudocode

START

1. Open the video camera.

2. WHILE camera is running:

a. Capture a video frame.

b. Resize the frame if necessary  
to improve processing speed.

c. Convert the frame to grayscale.

d. Apply Gaussian blur  
to reduce image noise.

e. Apply Canny edge detection.

f. Display the detected edges.

g. IF exit key is pressed:

BREAK

END IF

3. Release the camera.

4. Close all windows.

END

### **Modules**

**Camera → Resize → Grayscale → Gaussian Blur → Canny → Display**

### **Justification**

- Resizing reduces computation.
- Grayscale reduces processing complexity.
- Gaussian blur removes noise before edge detection.
- Canny produces clear and thin edges.
- Continuous frame processing maintains real-time performance.